

Partie 1

Tester automatiquement avec Foundry

Le problème du test manuel

Dans Remix, pour vérifier le PaymentSplitter, il fallait :

déployer → envoyer 10 ETH → cliquer pendingPayment → cliquer release → relire...

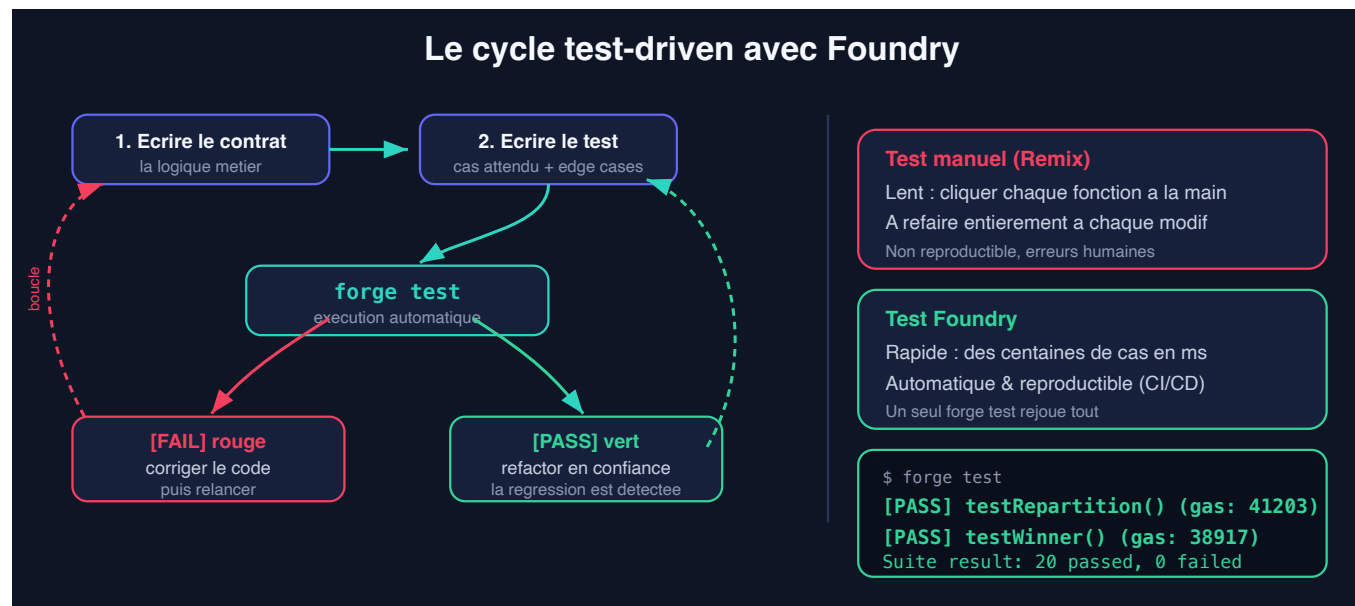
À chaque modification du contrat, **on recommence tout, à la main.**

✗

Et on ne teste en pratique que **le cas qui marche** — jamais les dizaines de cas d'erreur (revert attendu, débordement, accès interdit, cas limite).

“ Un contrat gère de l'argent. Un cas non testé = une faille potentielle en production. ”

La boucle test-driven



Le test devient un **filet de sécurité** : on modifie le contrat sans crainte de régression.

Foundry : tester en Solidity

Foundry exécute des tests écrits... en Solidity. Rapide (moteur en Rust).

```
import "forge-std/Test.sol";
import "../src/Exercice_3_PaymentSplitter.sol";

contract SplitterTest is Test {
    PaymentSplitter splitter;

    function setUp() public {          // exécuté AVANT chaque test
        address[] memory payees = new address[](3);
        // ... alice/bob/carol, shares 50/30/20
        splitter = new PaymentSplitter(payees, shares);
    }

    function testRepartition() public {
        (bool ok,) = address(splitter).call{value: 10 ether}("");
        assertEq(splitter.pendingPayment(alice), 5 ether); // l'assertion
    }
}
```

Les cheatcodes : simuler tout un scénario

Cheatcode	Rôle
<code>vm.prank(addr)</code>	le prochain appel vient de <code>addr</code> (simule un autre user)
<code>vm.expectRevert(Err.selector)</code>	le test échoue si la fonction NE revert PAS
<code>vm.deal(addr, 1 ether)</code>	donne de l'ETH à une adresse
<code>vm.warp(timestamp)</code>	avance l'horloge (tester un délai)
<code>assertEq</code> <code>assertGt</code> <code>assertApproxEqAbs</code>	assertions

```
vm.prank(alice); // alice a 0 VOTE
vm.expectRevert(VotingToken.NotEnoughTokens.selector);
token.createProposal("..."); // DOIT revert, sinon test rouge
```

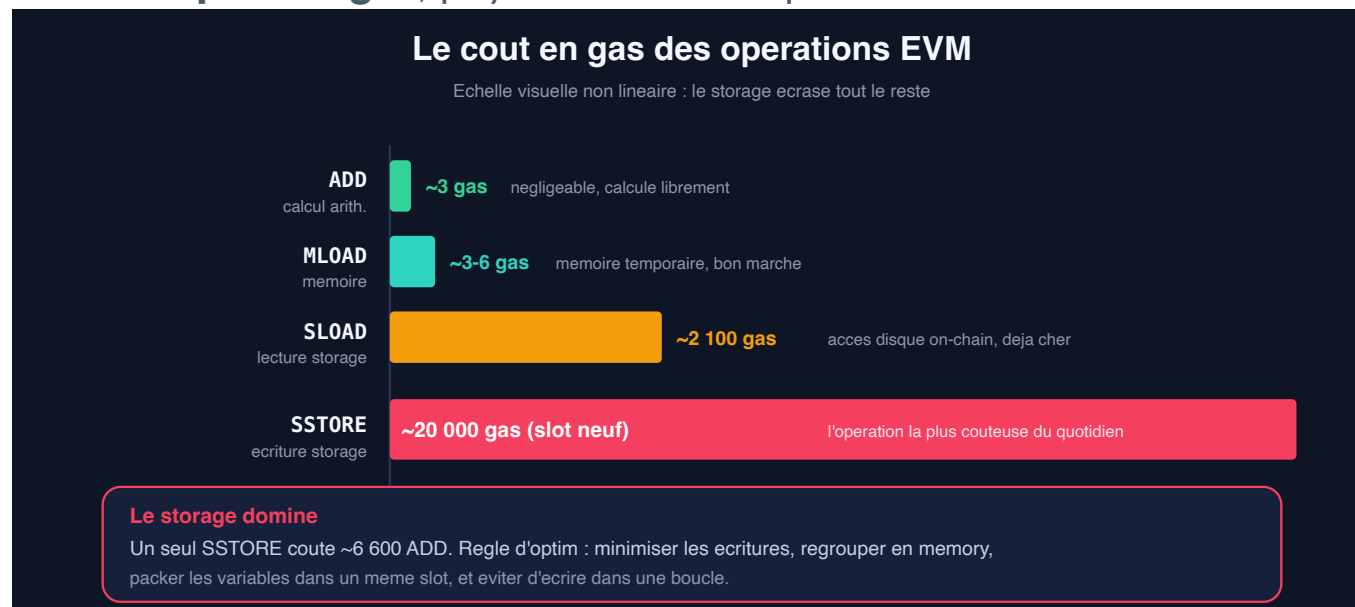
“ On teste autant les **échecs** que les succès. C'est là qu'est la vraie valeur. ”

Partie 2

Le gas : mesurer & optimiser

Pourquoi le gas compte

Chaque opération de l'EVM a un **prix en gas**, payé en ETH réel par l'utilisateur.



Le storage domine – l'ordre de grandeur

Opération	Coût indicatif	Rapport vs ADD
ADD / comparaison	~3 gas	×1
MLOAD (mémoire)	~3-6 gas	×2
SLOAD (lire storage)	~2 100 gas	×700
SSTORE (écrire storage)	~20 000 gas	⚠ ×6 600

“ Conséquence directe : **on écrit le storage le moins possible.**

Une variable lue 5× dans une boucle ? On la met en mémoire **une fois.**

”

La version naïve

×

```
// relit + réécrit `total` (storage) à CHAQUE tour de boucle
function sumAndStore(uint256[] calldata input) external {
    for (uint256 i = 0; i < input.length; i++) {
        values.push(input[i]);
        total = total + input[i];    // SLOAD + SSTORE × N itérations
    }
}
```

“ Chaque tour relit

input.length

et fait un aller-retour storage sur

total

.

Sur un tableau de 100 éléments, c'est **100 écritures** au lieu d'une.

”

La version optimisée



```
function sumAndStore(uint256[] calldata input) external {
    uint256 len = input.length;           // longueur en cache (1 lecture)
    uint256 acc = total;                   // 1 SLOAD
    for (uint256 i = 0; i < len; ) {
        values.push(input[i]); acc += input[i];
        unchecked { ++i; }                // pas de check overflow inutile
    }
    total = acc;                           // 1 SSTORE en tout
}
```

“ Même résultat, mais l'accumulation se fait **en mémoire** → une seule écriture finale.

”

Mesurer :

```
forge test --gas-report
```

GasOptimized · sumAndStore	180 416 gas
GasNaive · sumAndStore	183 389 gas

“ On ne devine pas le gas : on le **measure**.

Démo :

```
contracts/Demo05_GasComparison.sol
```

.

”

La boîte à outils d'optimisation

- accumuler en **mémoire**, écrire le **storage** une seule fois
- **caler**
`array.length`
hors de la boucle
- `calldata`
plutôt que
`memory`
en entrée d'
`external`
- `unchecked { ++i; }`
quand l'overflow est impossible
- **packing** : regrouper des variables dans un même slot de 32 octets
- `immutable`
/
`constant`
quand la valeur ne change jamais

Partie 3

Oracles — connecter le monde réel

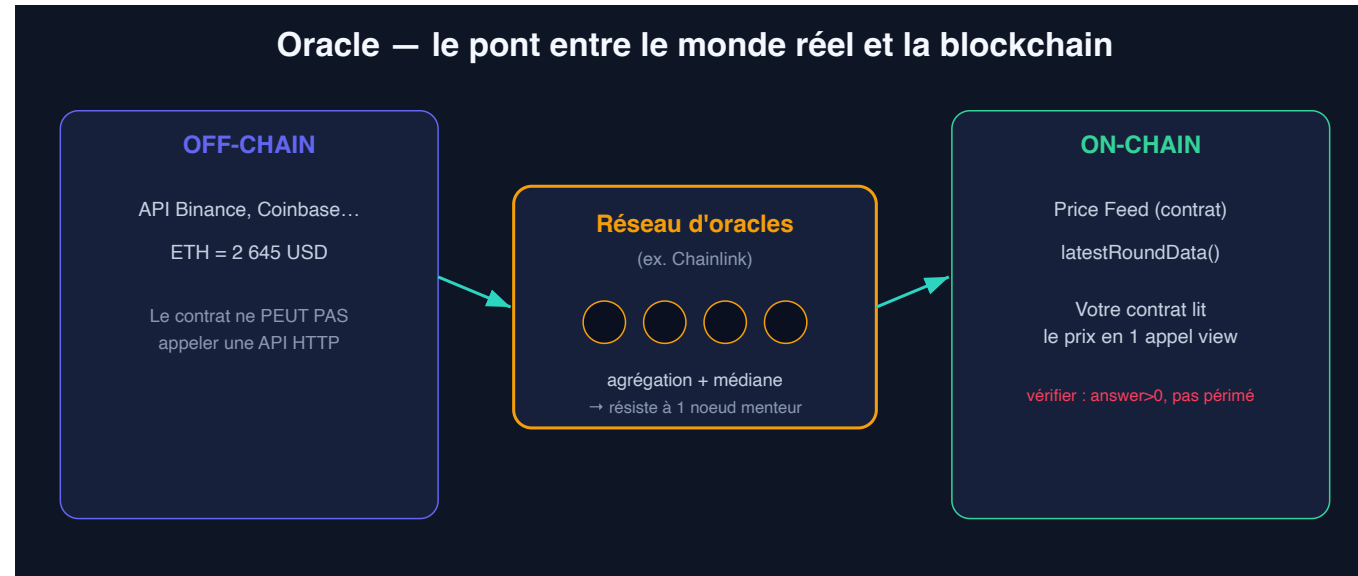
Un contrat est aveugle

Un smart contract est **déterministe et isolé** – il doit donner le **même résultat sur tous les nœuds**. Donc :

✗ Impossible nativement	Pourquoi
appeler une API HTTP	non déterministe, chaque nœud aurait une réponse
connaître le prix ETH/USD	donnée externe au registre
générer un vrai hasard	tout est reproductible sur l'EVM
lire la météo, un score sportif	hors-chaîne

“ Pourtant la DeFi a besoin du **prix**, l'assurance du **résultat d'un vol**, la loterie d'un **vrai aléa**. La solution : un **oracle**. ”

Comment marche un oracle



Lire un price feed Chainlink — en sécurité

```
interface AggregatorV3Interface {
    function latestRoundData() external view
        returns (uint80, int256 answer, uint256, uint256 updatedAt, uint80);
}

function getLatestPrice() public view returns (int256) {
    (, int256 answer, , uint256 updatedAt, ) = priceFeed.latestRoundData();
    if (answer <= 0) revert InvalidPrice(); // anomalie
    if (block.timestamp - updatedAt > 3600) revert StalePrice(); // périmé
    return answer;
}
```

“ **Sécurité oracle** = toujours vérifier que le prix est **positif** et **récent**.

Une manipulation d'oracle (flash loan + prix faussé) = des millions perdus —
c'est l'une des 6 vulnérabilités vues au J3. Démo :

Demo06_PriceConsumer.sol

”

L'aléa truquable – rappel du mini-CTF (J3)

```
//
X
Challenge 4 du CTF : loterie manipulable
uint256 idx = uint256(keccak256(abi.encodePacked(
    block.timestamp, block.prevrando
))) % players.length;
```

Tout ce qui est on-chain est **observable ou influençable** par un validateur :

block.timestamp

,

block.prevrando

,

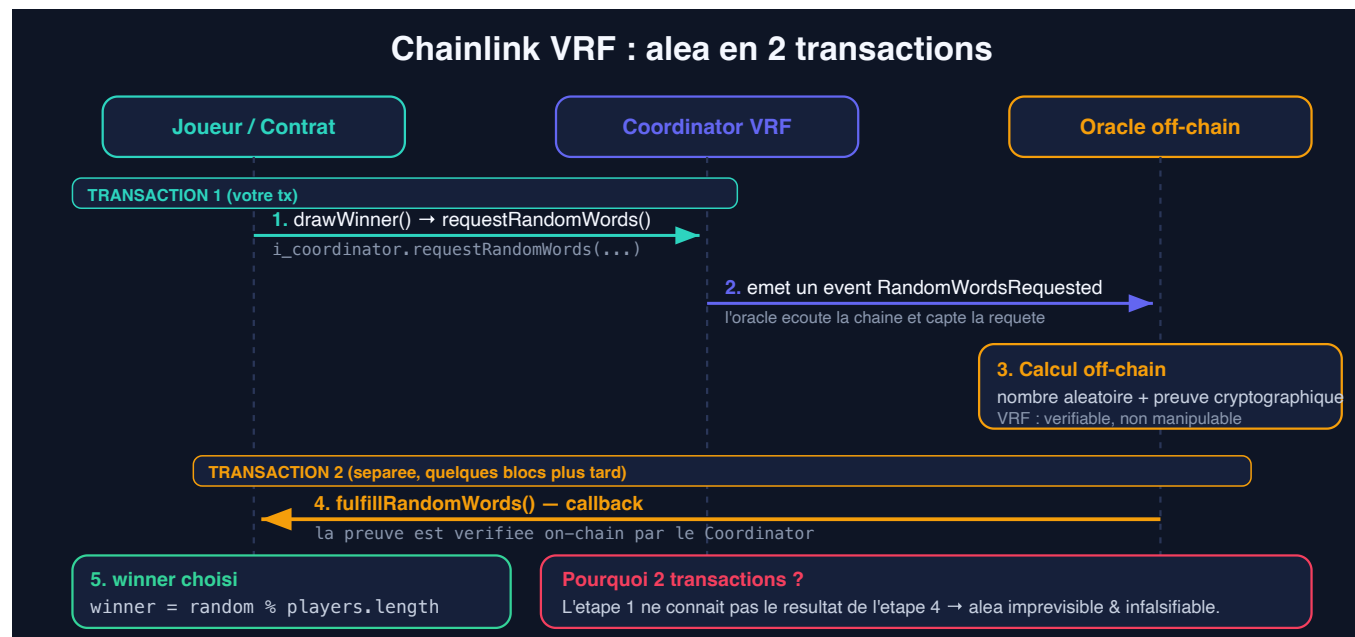
blockhash

...

“ Un validateur peut **choisir** de publier (ou non) le bloc selon le gagnant.
On ne tire **jamais** une valeur financière de cet « aléa ».

”

La vraie solution : Chainlink VRF



L'aléa arrive dans une **transaction séparée** + une **preuve** → imprévisible et vérifiable.

VRF en code (squelette)

```
contract FairLottery is VRFConsumerBaseV2 {
    function drawWinner() external onlyOwner {
        state = CALCULATING;
        requestId = COORDINATOR.requestRandomWords(/* keyHash, subId, ... */);
    }

    // callback appelé par l'oracle DANS UNE AUTRE TX
    function fulfillRandomWords(uint256, uint256[] memory words) internal override {
        uint256 winner = words[0] % players.length;
        // ... transfert de la cagnotte, reset, state = OPEN
    }
}
```

“ 

Exercice 7 — réécrire la loterie du CTF avec VRF.

Sujet :

sujets/Exercice_7_LoterieVRF.md

(mock VRF pour tester en local).

”

Partie 4

Contrats upgradeables

Le paradoxe

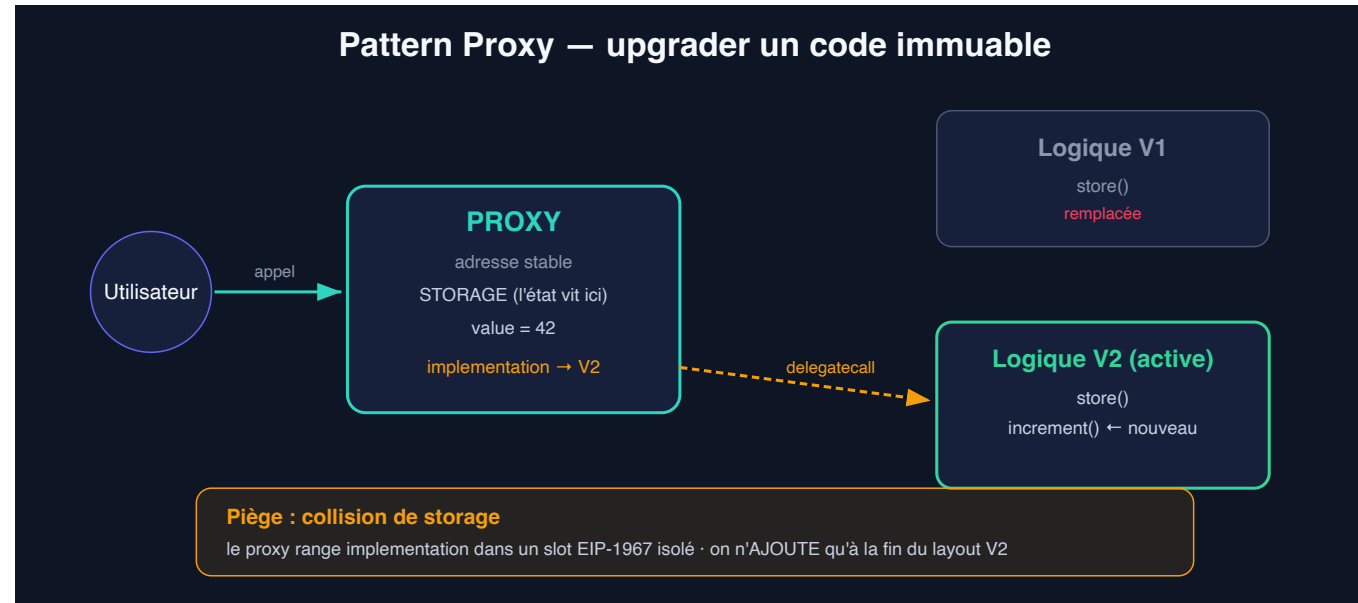
“ « Un smart contract est **immuable**. »

... mais s'il a un **bug critique** ? ou qu'il faut une **nouvelle fonctionnalité** ? ”

On ne peut pas modifier le bytecode déployé. La solution : **séparer**

Composant	Rôle	Mutable ?
Proxy	adresse stable + stockage de l'état	non (mais pointe vers...)
Implémentation	la logique (les fonctions)	remplaçable

Le pattern Proxy



Le proxy **délègue** (

`delegatecall`

) l'exécution à l'implémentation, **dans son propre storage**.

Le piège : la collision de storage



delegatecall & le slot EIP-1967

```
fallback() external payable {
    address impl = _getSlot(_IMPL_SLOT);    // slot dédié, hors slots 0,1,2...
    assembly {
        calldatacopy(0, 0, calldatasize())
        let r := delegatecall(gas(), impl, 0, calldatasize(), 0, 0)
        returndatacopy(0, 0, returndatasize())
        switch r case 0 { revert(0, returndatasize()) }
                default { return(0, returndatasize()) }
    }
}

// implementation rangée à un slot "impossible à percuter" :
bytes32 constant _IMPL_SLOT =
    0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc;
```

“ Sans ce slot dédié →

`store()`

écrase l'adresse d'implémentation → **contrat mort**.

”

Upgrade en pratique (V1 → V2)

1. Déployer BoxV1, puis Proxy(BoxV1)
2. store(42) via le proxy → value = 42 (état DANS le proxy)
3. Déployer BoxV2 (ajoute increment())
4. proxy.upgradeTo(BoxV2)
5. value vaut TOUJOURS 42 + increment() existe maintenant



Règle de survie du storage layout :



- on **ajoute** de nouvelles variables **à la fin**



- on ne **réordonne** jamais, on ne **retire** jamais, on ne **retype** jamais

“ Démonstrée :

Demo07_UpgradeableBox.sol

— état conservé après upgrade



En prod : OpenZeppelin **UUPS** / **Transparent Proxy** + plugins Foundry/Hardhat.

”

Partie 5

NFT & marketplace — Exercice 5

ERC-721 : les tokens non-fongibles (NFT)

Là où l'ERC-20 gère des jetons **interchangeables**, l'ERC-721 gère des objets **uniques** : chaque token a un **id** et un **propriétaire** distincts.

```
import "@openzeppelin/contracts/token/ERC721/ERC721URIStorage.sol";

contract GameItem is ERC721URIStorage {
    uint256 private _nextId;
    constructor() ERC721("Game Item", "ITEM") {}

    function mint(address to, string memory uri) external returns (uint256 id) {
        id = _nextId++;
        _safeMint(to, id);
        _setTokenURI(id, uri); // metadata (image, attributs) → souvent IPFS
    }
}
```

“ D  mo :

continue/contracts/Demo04_SimpleNFT.sol

”

Le modèle approve / transferFrom

Pour qu'un **tiers** (la marketplace) déplace votre NFT, vous devez l'**autoriser** :

```
nft.approve(address(marketplace), tokenId); // un token précis  
nft.setApprovalForAll(address(marketplace), true); // toute la collection
```

Ensuite seulement la marketplace peut appeler :

```
nft.safeTransferFrom(seller, buyer, tokenId);
```

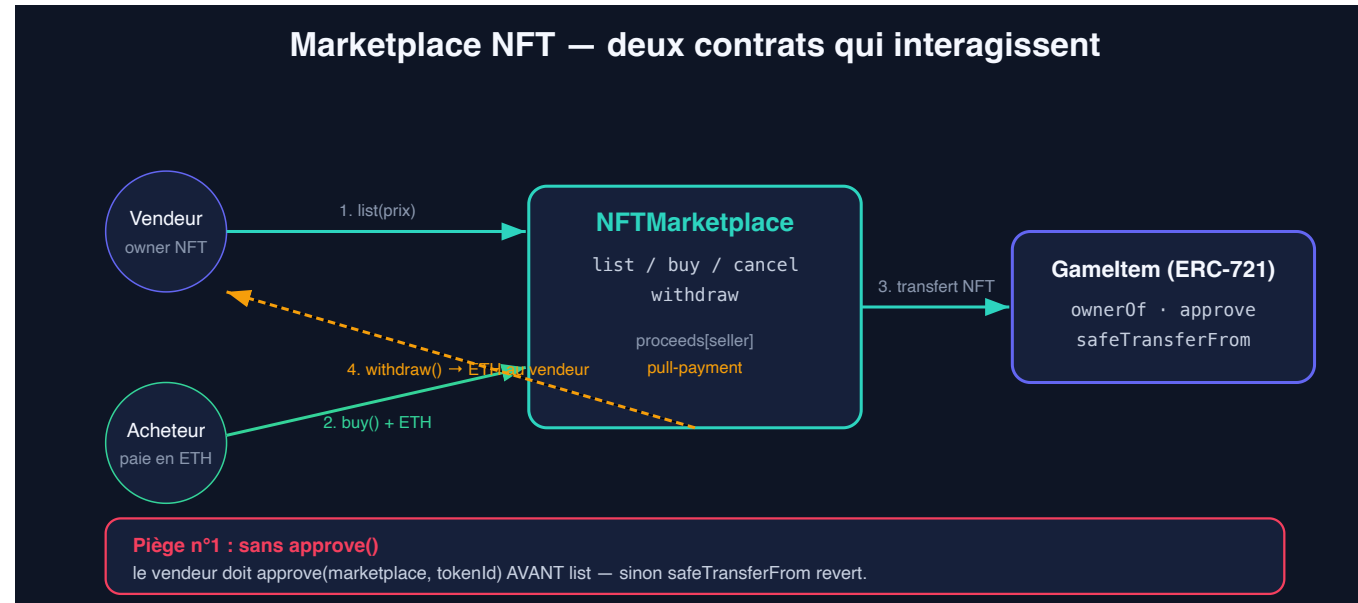
“ Sans approbation →

safeTransferFrom

revert. C'est le piège n°1 de l'exercice 5.

”

Exercice 5 — Marketplace NFT



Exercice 5 – l'énoncé

Faire **interagir deux contrats** : un NFT (ERC-721) + une place de marché.

Fonction	Comportement
<code>list(nft, id, price)</code>	owner + approuvé, prix > 0
<code>buy(nft, id)</code> <i>payable</i>	<code>msg.value == price</code> , transfère le NFT, crédite le vendeur
<code>cancel(nft, id)</code>	seulement le vendeur
<code>withdraw()</code>	pull-payment des gains du vendeur

- bonus : royalties, frais de plateforme,
`nonReentrant`

“Sujet :

`continue/sujets/Exercice_5_NFTMarketplace.md`

Squelette + corrigé :

`continue/contracts/Exercice_5_NFTMarketplace_*.sol`

• testé



(6/6)

Partie 6

DeFi en pratique : un mini-AMM

DEX : du carnet d'ordres à l'AMM

Une bourse classique fait correspondre **acheteurs et vendeurs** (carnet d'ordres).

On-chain, maintenir un carnet est **trop coûteux**. La DeFi a inventé l'**AMM** :

- une **pool** détient une réserve de deux tokens
- le **prix émerge** mécaniquement des réserves
- pas de contrepartie humaine : on échange **contre la pool**

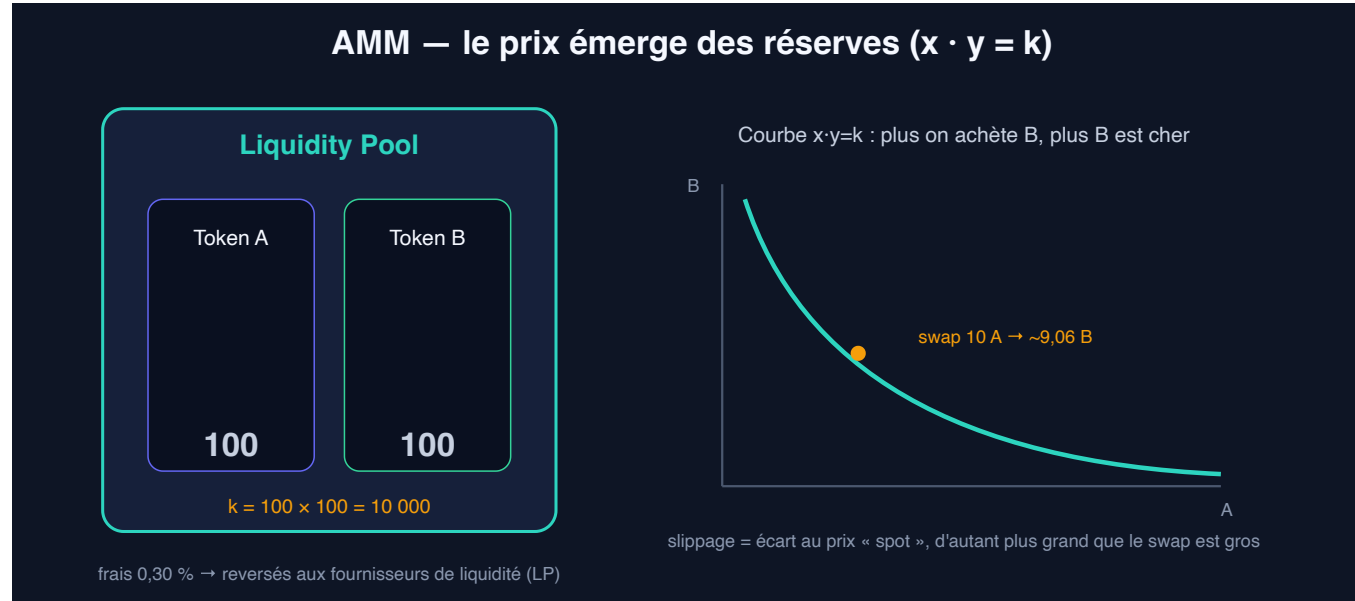
“ Uniswap, le plus gros DEX au monde, repose sur **une équation** :

$$x \cdot y = k$$

.”

”

La formule du produit constant



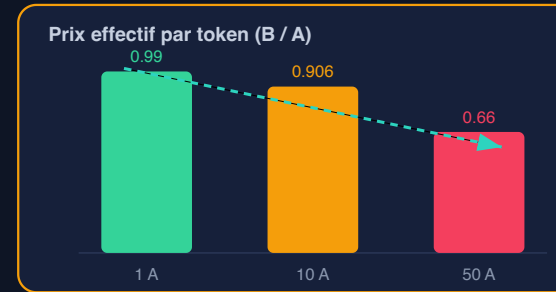
Le slippage : un gros swap rend moins

Slippage : pourquoi un gros swap rend moins

Pool 100 A / 100 B - $k = x * y = 10\,000$ (constant product)

Swap A -> B	Reçoit B	Prix / token
1 A	0.987 B	~0.99
10 A	9.06 B	~0.906
50 A	33.3 B	~0.66

Le 1er token achete proche du prix 1:1, chaque token suivant coute de plus en plus cher.



Conclusion

Plus le swap est gros par rapport a la taille de la pool, plus le slippage est fort.

Le prix bouge le long de la courbe $x*y=k$: acheter B fait monter son prix a chaque token retire de la reserve.

Le swap, en code

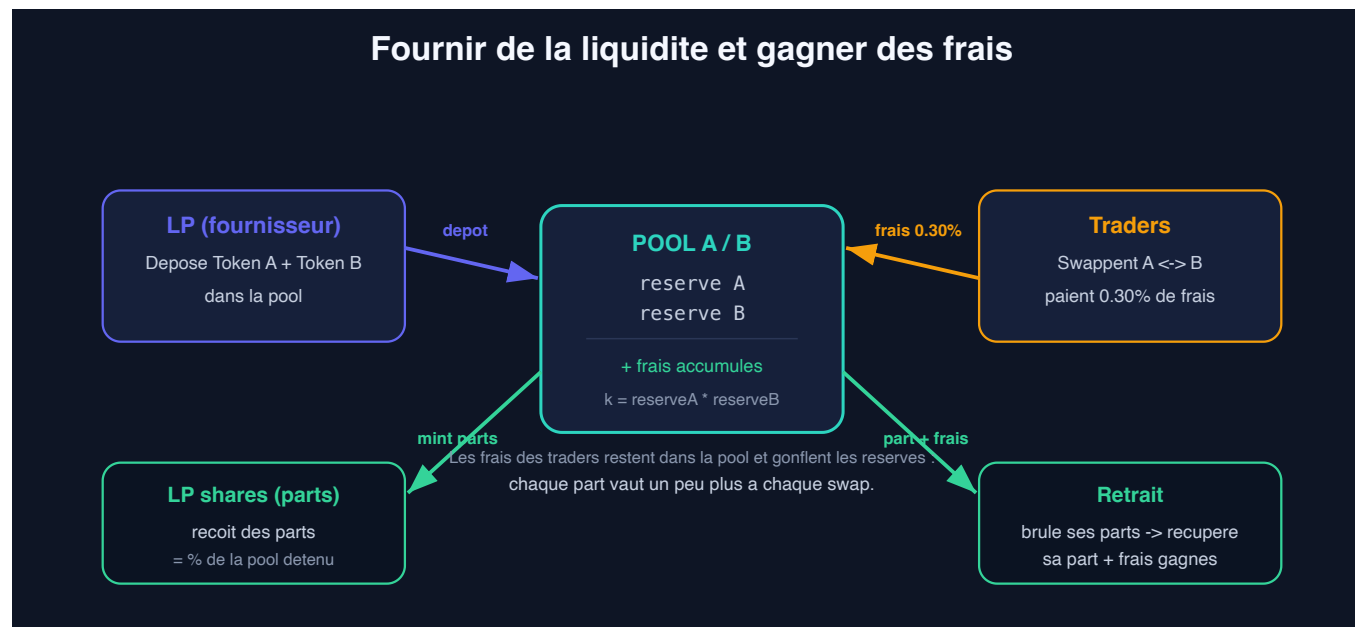
```
function getAmountOut(uint256 amountIn, uint256 reserveIn, uint256 reserveOut)
    public pure returns (uint256)
{
    uint256 amountInWithFee = (amountIn * (10000 - FEE_BPS)) / 10000; // 0,30 %
    return (reserveOut * amountInWithFee) / (reserveIn + amountInWithFee);
}
```

- sur une pool **100 / 100**, swap de **10 A** → on reçoit **~9,06 B** (pas 10 !)
- l'écart = **slippage** (le prix glisse) + **frais**
- **invariant**

k

: ne diminue jamais → garantit la solvabilité de la pool

Les fournisseurs de liquidité (LP)



“ 🖱️

Exercice 6 – compléter

MiniAMM

```
(
  addLiquidity
```

```
+
  swap
```

```
)
```

Partie 7

Account Abstraction (ERC-4337)

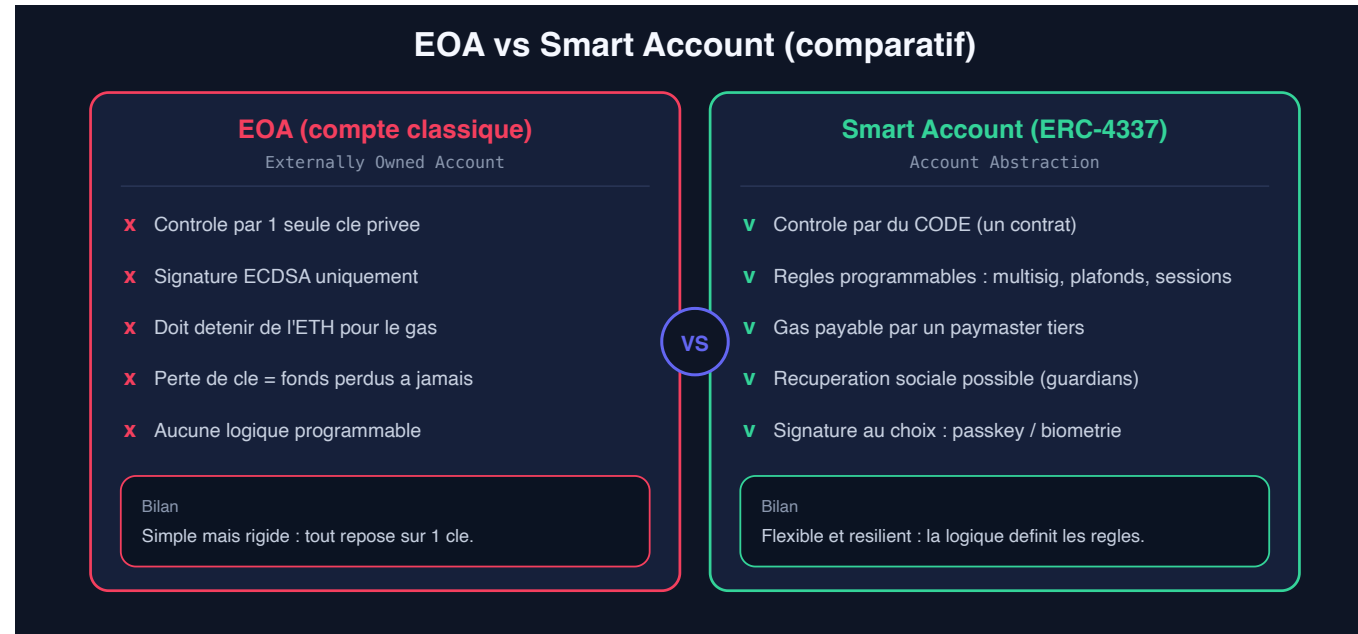
Le problème du wallet actuel (EOA)

Aujourd'hui un compte « normal » (EOA – *Externally Owned Account*) :

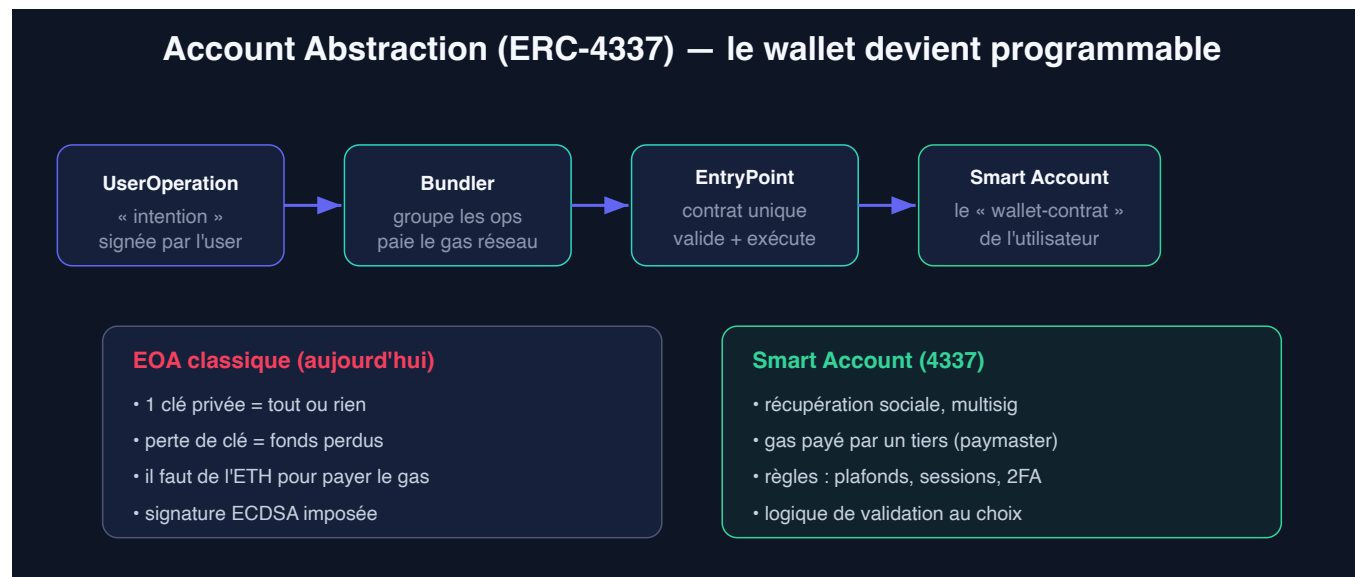
- **1 clé privée** = contrôle total • la perdre = **tout perdre**, sans recours
- obligation de détenir de l'**ETH** rien que pour payer le gas
- signature **ECDSA** imposée, **aucune** règle programmable

“ Pour l'adoption grand public, c'est un mur. **L'Account Abstraction** transforme le wallet en **smart contract programmable**. ”

EOA vs Smart Account



Le flux ERC-4337



Ce que ça débloque concrètement

Fonction	Exemple d'usage
Récupération sociale	retrouver l'accès via des « gardiens » (amis, autres devices)
Paymaster	un tiers paie le gas → l'utilisateur n'a pas besoin d'ETH
Sessions / plafonds	« ce jeu peut dépenser 10 € max sans re-signer »
Multisig natif	2 signatures sur 3 pour valider une opération
Signature au choix	passkey / biométrie au lieu de la <i>seed phrase</i>

“ Sans **hard fork** : tout passe par un contrat **EntryPoint** standardisé.
C'est la brique qui rapproche le Web3 de l'expérience d'une app classique.

”

Synthèse & ouverture

Ce que vous savez faire maintenant

Compétence	Outil / pattern
✓ tester un contrat automatiquement	Foundry, cheatcodes, gas-report
✓ mesurer et réduire le gas	storage, calldata, unchecked, packing
✓ lire un oracle en sécurité	price feed (périmé/positif), VRF pour l'aléa
✓ upgrader un contrat	proxy + slot EIP-1967, layout discipliné
✓ coder un AMM	$x \cdot y = k$, slippage, frais, liquidité
✓ situer l' Account Abstraction	ERC-4337, paymaster, smart account

Pour aller plus loin

Outil / sujet	Usage
Slither	analyse statique de vulnérabilités (gratuit, local)
Foundry fuzzing	tester avec des milliers d'entrées aléatoires
The Graph	indexer les events pour les requêter facilement
OpenZeppelin Defender	monitoring & opérations on-chain
Audits	Code4rena, Sherlock – la sécurité comme métier

“ La sécurité n'est jamais « finie » : **test + audit + monitoring** en continu.

”

Merci !

Questions ?

Utopios • Formation Blockchain

Place aux exercices 6 (Mini-AMM) & 7 (Loterie VRF)

